Squeezing Every FLOP: Single-GPU Training Efficiency Optimization for Vision-Language Models

A Case Study on SiQ-VL with NVIDIA Blackwell

Duo An
https://github.com/duoan/SiQ_VL

May 2026

Abstract

We present a systematic study of single-GPU training efficiency optimization for SiQ-VL, a vision-language model trained on NVIDIA RTX PRO 6000 Blackwell (96 GB VRAM). Through 15 iterations of measured optimizations—spanning dtype fixes, kernel fusion (Liger-Kernel, TileGym cuTile), data strategies (bucketing, packing), and batch scaling—we improve throughput from **4,674 to 107,367** real tokens/second: a **22.9× end-to-end speedup**. We document both successes and failures: vision feature caching (0% gain), Triton Flash Attention (slower than SDPA), FlexAttention packing (51% waste), and TileGym with variable shapes (5× regression). The final configuration achieves MFU≈22.7% on the 0.5B model, confirmed to be near the memory-bandwidth ceiling for this model scale.

Keywords: Vision-Language Models, Training Efficiency, GPU Kernel Optimization, cuTile, Liger-Kernel, Blackwell, Sequence Packing

arXiv: cs.LG, cs.CV
Code: https://github.com/duoan/SiQ_VL

Contents

1 Introduction

Training vision-language models (VLMs) efficiently on a single GPU is crucial for researchers with limited hardware budgets. While much attention has been paid to distributed training at scale, the *single-GPU efficiency frontier* determines the baseline throughput that distributed methods multiply.

This report documents the complete optimization journey of **SiQ-VL**, a compact VLM combining a SigLIP-2 vision encoder with a Qwen2.5 language model. We focus exclusively on engineering optimizations that preserve model quality (identical loss curves) while maximizing tokens processed per second.

1.1 Optimization Timeline

The optimization proceeded through 15 iterations over two model configurations:

1. **Phase A** (Iter 0–10): SigLIP2-so400m + Qwen2.5-1.5B (589M+494M params). Baseline 4,674 tok/s → 28,322 tok/s (**6.06**×).
2. **Phase B** (Iter 11–15): SigLIP2-base-224 + Qwen2.5-0.5B (92.9M+494M params). Focus on kernel optimization (TileGym cuTile). Baseline 21,673 tok/s → 107,367 tok/s (**4.95**×).

Combined end-to-end: **4,674 → 107,367 = 22.9**× (across model scales).

1.2 Contributions

1. A **reproducible benchmark** (`benchmark_real_efficiency.py`) that measures all metrics on actual training data.
2. Comprehensive evaluation of **Liger-Kernel vs TileGym cuTile**: TileGym is 30–41% faster with 50–64% less VRAM on Blackwell.
3. Demonstration that **length bucketing + large batch** renders sequence packing marginal (+4.6%) for uniform-length datasets.
4. Documentation of **6 failed optimizations** with root-cause analysis.
5. Evidence that 0.5B models are **memory-bandwidth bound** at 19–23% MFU, independent of kernel or data strategy.

2 System Setup

Table 1: Hardware and model configurations

Hardware	
GPU	NVIDIA RTX PRO 6000 Blackwell Server Edition
VRAM / Bandwidth	96 GB GDDR7 / 1.79 TB/s
Compute	188 SMs @ 2430 MHz, Peak BF16: 936 TFLOP/s
CUDA / Driver	CUDA 13.0, Driver 580.126
Model (Phase B — final configuration)	
Vision Encoder	SigLIP2-base-patch16-224 (92.9M, frozen)
Language Model	Qwen2.5-0.5B-Instruct (494.0M)
Projector	2-layer MLP + Pixel Shuffle (2.8M)
Total	589.7M parameters
Software	
Framework	PyTorch 2.9.1, Transformers 5.3.0
Kernels	TileGym (cuTile DSL), Liger-Kernel 0.8.0
Dataset	sharegpt4v/coco (50K samples)

3 Related Work

Vision-Language Model Architectures. The two-stage VLM training recipe (projector alignment → instruction tuning) was popularized by LLaVA [1] and LLaVA-1.5 [2]. Subsequent works scale this paradigm: InternVL [4] introduces dynamic resolution with progressive training at 6B+ scale; Qwen-VL [5] and Qwen2-VL [6] use three-stage pipelines with interleaved image-text pre-training; LLaVA-NeXT [3] explores dynamic high-resolution with AnyRes tiling; PaliGemma [7] and Phi-3-Vision [8] demonstrate that sub-4B VLMs can achieve competitive quality. SiQ-VL follows the LLaVA two-stage recipe using SigLIP-2 [9] as vision encoder and Qwen2.5 [10] as language backbone. Our contribution is orthogonal to architecture—we optimize *training throughput* for any VLM following this paradigm.

GPU Kernel Optimization. FlashAttention [11] and FlashAttention-2 [12] reduce attention memory from $O(N^2)$ to $O(N)$ via tiled IO-aware computation. Triton [13] provides a Python-based DSL for writing custom GPU kernels, enabling projects like Liger-Kernel [14] that fuse LLM training operators (RMSNorm, SwiGLU, RoPE, cross-entropy) in Triton. Unsloth [15] takes a similar approach for LoRA fine-tuning. On the vendor side, NVIDIA's cuTile DSL (part of CUTLASS 4.0) provides direct access to Blackwell-specific hardware: wgmma (warp-group matrix multiply), TMA (Tensor Memory Accelerator), and SM remapping. We show that cuTile kernels (via TileGym) outperform Triton-based Liger by 30–41% on Blackwell, establishing a clear hierarchy: vendor DSL > Triton > vanilla PyTorch on new architectures.

Mixed Precision and Memory Efficiency. Mixed-precision training [17] using FP16/BF16 is now standard. Gradient checkpointing [18] trades compute for memory. The "online softmax" trick [19] computes cross-entropy without materializing full logits; we extend this with a "grad-in-forward" pattern that computes ∇_h within the forward pass, avoiding storage of $B \times L \times V$ gradient tensors. Memory-efficient optimizers [20] like 8-bit Adam [21] and FlashOptim [22] (gradient release + fused state updates) further reduce optimizer memory; we benchmark FlashOptim and find marginal gains at the 0.5B scale.

Sequence Packing and Variable-Length Training. Packing multiple sequences into fixed-length bins to eliminate padding was introduced for BERT pre-training [23]. For causal LMs, block-diagonal attention masks [24] or `cu_seqlens`-based variable-length attention [11] maintain sample isolation within packed sequences. HuggingFace Transformers 4.57+ provides native packing support via position_id reset detection with FlexAttention [16] backends. Our contribution is quantifying the *diminishing returns* of packing: with effective bucketing at large batch sizes, padding waste is already <5%, making packing's throughput gain marginal (+4.6%).

Distributed Training Systems. Megatron-LM [25] pioneered tensor and pipeline parallelism. DeepSpeed ZeRO [26, 27] partitions optimizer states, gradients, and parameters across GPUs. PyTorch FSDP [28] integrates sharded data parallelism natively. These systems optimize multi-GPU communication and memory distribution. Our work addresses the orthogonal, often overlooked problem of *per-device* efficiency—the throughput floor that distributed methods multiply. Every technique we identify (batch scaling, shape quantization, kernel selection) benefits each GPU in a distributed setup independently.

Training Efficiency Metrics. PaLM [29] formalized Model FLOPs Utilization (MFU) as the ratio of achieved to theoretical peak throughput. Chinchilla [30] establishes compute-optimal scaling laws. We contribute MFU measurements for sub-1B VLMs on Blackwell—a previously uncharacterized regime—and show the memory-bandwidth ceiling at 19–23% MFU is structural for small models, not a software limitation.

4 The Optimization Journey

Each experiment follows the structure: **Hypothesis** → **Experiment** → **Finding** → **Conclusion** → **Lesson Learned**.

4.1 Iteration 0 — Baseline Measurement

Hypothesis. We expect the baseline to be limited by attention compute (SDPA vs FlashAttention) and unoptimized dataloader transfers.

Experiment. Run 20 profiled training steps with `torch.profiler` (B=4, grad_accum=4, bf16, gradient_checkpointing). Inspect CUDA kernel dispatches.

Finding. 49.96% of CUDA time is `cutlass_80_simt_sgemm`—an *FP32* GEMM kernel. Despite `bf16=True` in TrainingArguments, the actual model weights are in FP32 on device. `from_pretrained()` default loads FP32; HF Trainer's autocast only wraps the forward call, not the model parameters.

Metric	Value
Tokens/sec	**4,674**
Step time	310.7 ms
Peak VRAM	19.27 GB
Top CUDA kernel	`cutlass_sgemm` (FP32!) 33.6%

Conclusion. The system is running FP32 GEMM on a GPU with 936 TFLOP/s BF16 tensor cores. This is the single largest performance bug—not attention, not data loading.

Lesson. *Always inspect actual kernel dispatches in the profiler trace. Config flags (`bf16=True`) do not guarantee the intended dtype reaches the hardware.*

4.2 Iteration 1 — Fix FP32 + Vision `no_grad`

Hypothesis. Loading with `torch_dtype=torch.bfloat16` will force BF16 tensor-core GEMM dispatch. Wrapping the frozen vision encoder in `torch.no_grad()` will eliminate its autograd graph, saving time and memory.

Experiment. Modify `get_stage1_model_and_processor` to pass `torch_dtype=torch.bfloat16`; wrap vision forward in `torch.no_grad()`.

Metric	Before	After	Δ
Tokens/sec	4,674	**14,366**	**+207%**
Step time (ms)	310.7	101.1	–67%
Peak VRAM (GB)	19.27	11.54	–40%

Finding.

Conclusion. A **3.07× speedup** from a 2-line change. BF16 GEMM dispatches confirmed via profiler (`cutlass_bf16_tensor_op_gemm`).

Lesson. *The first profiler run is worth more than a week of kernel optimization. This single fix delivered more speedup than all subsequent iterations combined.*

4.3 Iteration 2 — Liger-Kernel Integration

Hypothesis. Liger-Kernel's fused operators (Linear-CE, RMSNorm, SwiGLU, RoPE) will reduce kernel launch overhead and eliminate the large $(B, L, 151936)$ logits tensor, improving both speed and memory.

Experiment. Integrate via `apply_liger_kernel_to_qwen2(rope=True, fused_linear_cross_entropy=Tr`
`rms_norm=True, swiglu=True)`.

Metric	Before (Iter 1)	After (Liger)	Δ
Tokens/sec	14,366	11,070	**–25%**
Peak VRAM (GB)	11.54	**6.78**	**–41%**

Finding. Speed is *worse*. In Stage 1 the LLM is frozen—no backward runs through the LLM layers, so fused CE backward savings never materialize. Triton JIT compilation adds 100ms spikes. However, VRAM drops 41% because fused CE never materializes full logits.

Conclusion. Keep for VRAM savings (enables larger batches later), but Liger is **not a speed win for Stage 1 projector-only training**.

Lesson. *Kernel fusion benefits depend on which modules have gradients. Stage 1 (frozen LLM) gains nothing from LLM-internal fusion. Always profile the actual training stage, not just micro-benchmarks.*

4.4 Iteration 3 — Vision Acceleration Investigation

Hypothesis. Three candidate approaches to further speed up Stage 1: (a) pre-cache SigLIP features to skip vision forward entirely; (b) `torch.compile` on the frozen vision encoder; (c) simply increase batch size to better utilize the GPU.

Experiment. Implement all three independently and measure tok/s.

Table 2: Iteration 3: Three approaches compared

Approach	Tok/s	VRAM	Verdict
Baseline (Iter 2, B=4)	11,070	6.8 GB	—
(a) Vision feature caching	11,070	8.6 GB	Worse (+27% VRAM, 0% speed)
(b) torch.compile on vision	11,400	6.8 GB	Negligible (+3%)
(c) **Batch 4→16**	**20,284**	**16.5 GB**	**Winner (+83%)**

Finding. Vision caching fails because SigLIP forward takes only 5 ms/tile on Blackwell (too fast to benefit from caching), while loading cached tensors (9.4 MB/sample) from CPU→GPU via DataLoader is slower. torch.compile has no opportunity because SDPA already dispatches to flash kernels and shapes are dynamic.

Conclusion. The GPU is **underutilized at B=4**. Increasing batch size is the highest-leverage knob: 4× more tokens per step, only 2.4× more VRAM.

Lesson. *Don't cache what's already fast. On modern GPUs with high tensor-core throughput, re-computing vision features is cheaper than transferring large pre-computed tensors over PCIe/memory bus. Always compare compute cost vs transfer cost.*

4.5 Iteration 4 — Offline Image Preprocessing (Planned, Superseded)

Hypothesis. CPU-bound image preprocessing (PIL resize + normalize + tile) is the DataLoader bottleneck. Pre-processing all images to tensor shards on disk would let the DataLoader only tokenize text and load pre-computed tensors via mmap.

Experiment. Planned but not executed.

Finding. Iteration 3 revealed that vision compute (SigLIP forward) is only 5 ms/tile on Blackwell—far faster than expected. The DataLoader is NOT the bottleneck; the GPU is simply underutilized at B=4. Pre-processing would save ∼5 ms/step on a 270 ms step (1.9%), making it not worth the infrastructure complexity.

Conclusion. **Superseded** by batch size scaling (Iter 3). The planned offline preprocessing infrastructure was never needed because vision compute is not the bottleneck on modern GPUs.

Lesson. *Measure before building infrastructure. We planned an entire caching system (scripts, datasets, collators) before discovering that the "slow" component takes only 5 ms. Batch scaling gave 83% improvement with zero infrastructure.*

4.6 Iteration 5 — Length Bucketing

Hypothesis. Intra-batch padding wastes 17.4% of compute. Sorting samples by length (HF `group_by_length`) will group similar-length sequences, reducing padding to near zero.

Experiment. Add `lengths` property to VQADataset with heuristic estimation (3.5 chars/token + overhead). Enable `group_by_length=True`.

Finding. Padding waste drops from 17.4% to 4.1% (–13pp). However, tok/s improves only **3.7%** (20,284 → 21,026).

Conclusion. Bucketing is a *free* optimization (no code complexity, no memory cost), but the throughput gain is modest because sharegpt4v/coco has low length variance (CoV=11.5%, range 260–502 tokens).

Lesson. *Padding waste ≠ throughput loss. The GPU processes padded tokens at the same speed as real tokens (same GEMM size). The "waste" is that you computed tokens that don't contribute to learning, but wall-clock time per step barely changes if the max sequence length in each batch stays similar.*

4.7 Iteration 6 — First Packing Attempt (Failed)

Hypothesis. Concatenating sequences into fixed-length packed bins with a 4D block-diagonal attention mask should eliminate all padding.

Experiment. Implement packing with explicit `attention_mask` (4D dense tensor, shape $B \times 1 \times N \times N$) under SDPA backend.

Finding. SDPA detects the non-trivial 4D mask and falls back to the *math kernel* (naive $O(N^2)$ materialized attention)—orders of magnitude slower than the flash/efficient kernels used for simple causal masks. Training step time increased 3×.

Conclusion. Reverted. SDPA cannot efficiently handle arbitrary 4D masks. A different attention backend is needed for packing.

Lesson. *Attention backend selection is not just about "flash vs SDPA"—it's about what mask shapes each backend supports efficiently. SDPA's flash path only handles causal masks; any custom mask triggers the slow fallback. This motivated the move to FlexAttention in Iter 10.*

4.8 Iteration 7 — torch.compile Replaces Liger

Hypothesis. `torch.compile(mode="max-autotune-no-cudagraphs")` applied to the full model should fuse ops across the entire graph—outperforming Liger's targeted patches. The two may be combinable for maximum effect.

Experiment. Attempt compile with Liger; when that crashes (illegal memory access), try compile *without* Liger as a replacement.

Config	Tok/s	VRAM	vs Iter 5
Liger + compile	CRASH (illegal memory access)		
Liger only (Iter 5)	21,026	15.8 GB	—
Compile only (no Liger)	**27,352**	20.7 GB	**+30.1%**

Finding.

Conclusion. Liger and torch.compile are **mutually exclusive**—Liger's Triton patches confuse `torch._dynamo` graph capture. Compile alone gives 30% speedup over Liger by optimizing the entire computational graph (attention + MLP + embedding).

Lesson. *Whole-graph compilation > targeted operator fusion for this workload. But there's a trade-off: compile loses Liger's 41% VRAM savings (no fused CE). Choose based on whether you're memory-constrained or compute-constrained.*

4.9 Iteration 8 — Batch Tuning Under torch.compile

Hypothesis. With compile reducing overhead, increasing B from 16 to 24–32 should further improve GPU utilization and tok/s.

Experiment. Sweep B $\in \{16, 20, 24, 28, 32\}$ under torch.compile. Also test `reduce-overhead` mode (cudagraphs) and `pad_to_multiple_of=64`.

Config	Tok/s	VRAM	Δ	Note
B=16, compile	27,108	23.7 GB	—	
B=20, compile	**28,032**	29.8 GB	+3.4%	Optimal
B=24, compile	27,440	38.5 GB	+1.2%	Diminishing returns
B=32, compile	OOM	—	—	
`reduce-overhead`	22,000	—	–20%	Dynamic shapes kill cudagraphs
`pad_to_multiple_of=64`	25,300	—	–6.6%	Extra padding wastes compute

Finding.

Conclusion. Diminishing returns above B=20. cudagraphs is **counterproductive** for VLMs (dynamic shapes from variable image tiles cause frequent graph re-capture). This workload is at its single-GPU ceiling for the 1.5B model: MFU≈23%.

Lesson. *torch.compile's "reduce-overhead" mode (cudagraphs) assumes static shapes. VLMs with variable-resolution images violate this assumption fundamentally. Don't force static-shape optimizations on inherently dynamic workloads.*

4.10 Iteration 9 — DataLoader Optimization (Incremental)

Hypothesis. Increasing DataLoader workers and using a CUDA prefetch stream should overlap data transfer with GPU compute, reducing idle time between steps.

Experiment. Test num_workers $\in \{0, 2, 4, 8, 16\}$, add `pin_memory=True`, implement a CUDA prefetcher that pre-transfers the next batch to GPU on a separate stream.

Finding. num_workers=8 is optimal (+2% over workers=4). The CUDA prefetcher showed no measurable gain because HuggingFace Trainer already uses `pin_memory` and the H2D transfer (<1 ms for our batch sizes) is fully overlapped with GPU compute at B=16+.

Conclusion. Marginal. Kept num_workers=8 as default.

Lesson. *DataLoader optimization only matters when data loading is the bottleneck. With B≥16, each training step takes 200+ms of GPU compute, while data loading takes <5 ms. The GPU is never waiting for data. Focus optimization effort on the actual bottleneck (GPU compute efficiency).*

4.11 Iteration 10 — Sequence Packing + FlexAttention

Hypothesis. Even after bucketing, 10% padding remains on the mixed 6-subset dataset (length variance σ=400). Packing multiple short samples into fixed-length bins (N=1536) with FlexAttention block-diagonal masks should eliminate padding entirely.

Experiment. Implement `PackingCollator` with first-fit-decreasing bin-packing. Use HuggingFace's native packed-sequence detection via `position_ids` resets + `attn_implementation="flex_attention"`.

Config	Tok/s	Step (ms)	VRAM	Padding
Baseline (bucketed, SDPA)	25,561	192	41.4 GB	10.2%
Packing + FlexAttn	**26,787**	297	34.0 GB	∼2%

Finding. Only **+4.8% throughput** despite eliminating 80% of padding waste. The step time actually *increased* (192→297ms) because packed sequences are longer (N=1536 vs avg 456), increasing the attention quadratic term. The gain comes from processing more samples per step.

Conclusion. Packing is a modest win (+4.8% tok/s, –18% VRAM) on this dataset. The throughput gain is limited because bucketing already captured most length variance.

Lesson. *Packing eliminates padding tokens but creates longer sequences, which have their own cost (quadratic attention). The net gain depends on the ratio avg_sample_length/bin_size. When samples are short (∼350 tok) relative to the bin (1536), each bin holds only 3–4 samples with 30% wasted space—defeating the purpose. Bin size should be ∼2–3× average sample length.*

4.12 Iteration 11 — TileGym FA4: Blackwell-Native Attention

Hypothesis. NVIDIA's cuTile DSL can generate attention kernels that access Blackwell-specific features (wgmma, TMA, SM remapping) inaccessible from Triton. TileGym's pre-built FA4 with native GQA should outperform both SDPA and our custom Triton attention (which failed in an earlier attempt).

Experiment. Replace SDPA in the text model's attention layers with TileGym FA4 (`tilegym.ops.cutile.exp`
Benchmark kernel-level and end-to-end.

Table 3: TileGym FA4 vs SDPA (end-to-end Stage 1 step time)

Backend	N=512	N=1024	N=1536	Why faster
SDPA	69.2 ms	82.1 ms	84.3 ms	Must expand K/V 7× (GQA)
TileGym FA4	64.9 ms	75.2 ms	75.0 ms	Native GQA in-kernel
Speedup	**1.07**×	**1.09**×	**1.12**×	Grows with seq len

Finding. (Note: a prior attempt with our own Triton Flash Attention failed—*slower* than SDPA because Triton cannot emit wgmma/TMA instructions for Blackwell.)

Conclusion. FA4 provides 7–12% E2E speedup in Stage 1 (where attention is forward-only). The gain increases with sequence length due to the quadratic attention term.

Lesson. *On new GPU architectures, vendor-provided kernel libraries (cuTile/TileGym) outperform portable solutions (Triton) because they can access architecture-specific instructions. Don't waste time writing custom Triton attention for Blackwell—use TileGym's production kernel.*

4.13 Iteration 12 — Stage 2 Readiness (cuTile Forward Doesn't Help Backward)

Hypothesis. TileGym FA4 gave 7–12% speedup in Stage 1 (forward-only attention). In Stage 2 (unfrozen LLM with full backward), using FA4 for forward + SDPA for backward (`cutile_training` backend) should still provide a net speedup.

Experiment. Benchmark Stage 2 (Full FT + grad_ckpt) with SDPA vs `cutile_training` (FA4 fwd + SDPA bwd via `autograd.Function` wrapper). Also benchmark LoRA vs Full FT across sequence lengths.

Table 4: cuTile_training vs SDPA for Stage 2 (unfrozen LLM)

Config	SDPA	cuTile+SDPA_bwd	Speedup
Full FT, N=512	272.3 ms	276.8 ms	0.98×
Full FT, N=1024	290.3 ms	292.6 ms	0.99×
Full FT, N=2048	333.6 ms	338.7 ms	0.99×
LoRA r=16, N=512	102.1 ms	112.4 ms	0.91×

Finding. The `autograd.Function` Python overhead (save/restore tensors, dispatch) cancels out the FA4 forward gain. With unfrozen LLM, the backward pass dominates (∼70% of step time)—a 12% faster forward contributes only ∼3.6% E2E, which is eaten by the wrapper overhead.

Also benchmarked: `flash-attn-4` (Dao-AILab) install is **blocked**—requires CUDA 13.1+ (system has 13.0).

Conclusion. Use plain SDPA for Stage 2. cuTile forward-only benefits *only* help when backward is never called (Stage 1 frozen LLM).

Lesson. *Optimizing a component that's <15% of step time (attention forward) while the rest (backward at 70%) is unchanged yields negligible E2E gains. Amdahl's Law is unforgiving: always target the dominant cost.*

4.14 Iteration 13 — TileGym Full Stack: ALL Qwen2 Ops

Hypothesis. If FA4 alone gives 7–12%, replacing *all* Qwen2 ops (RoPE, RMSNorm, SwiGLU, attention, CE loss) with cuTile equivalents should provide a larger cumulative speedup over Liger-Kernel.

Experiment. Apply `apply_tilegym_kernel_to_qwen2(use_cutile=True)` which patches all ops. Additionally, implement custom `FusedLinearCrossEntropy` using TileGym's `_ce_cutile` kernel with "grad-in-forward" pattern (compute ∇_h, ∇_W within the forward loop; backward becomes $O(1)$).

Table 5: TileGym full stack vs Liger-Kernel (Stage 2, grad_ckpt, B=4)

Seq Len	Liger tok/s	TileGym tok/s	Speed Δ	VRAM Δ
512	38.3K	49.7K	**+30%**	**−50%**
1024	46.5K	64.6K	**+39%**	**−59%**
2048	48.7K	68.7K	**+41%**	**−64%**

Finding. The VRAM reduction (50–64%) comes from the grad-in-forward CE pattern: logits ($B \times L \times V$) are never materialized; gradients are accumulated chunk-by-chunk (1024 tokens) during the forward pass.

Conclusion. TileGym's full stack is **both faster AND more memory-efficient** than Liger-Kernel on Blackwell. It is the correct kernel solution for this hardware.

Lesson. *cuTile's online softmax kernel (`_ce_online_kernel`) computes loss + grad in a single pass over vocab tiles. Combined with grad-in-forward (accumulate ∇_h per chunk, never store ∇_{logits}), this eliminates the largest memory bottleneck in LLM training: the logits tensor ($B \times L \times V = B \times L \times 151936$).*

4.15 Iteration 14 — Batch-Size Scaling + FlashOptim

Hypothesis. With TileGym full stack using only 5.6 GB VRAM (Stage 2, B=4), we have 90 GB of headroom. Scaling B to fill the GPU should increase throughput. Additionally, FlashOptim (Databricks) with gradient release can compress optimizer memory, enabling even larger batches.

Experiment. Sweep B from 32 to 896 (Stage 2, Full FT, grad_ckpt, TileGym, N=1024). Separately benchmark FlashAdamW vs standard AdamW.

Table 6: Batch scaling (Stage 2, TileGym full stack, N=1024)

B	ms/step	tok/s	VRAM	GPU Util
32	439	74,578	7.0 GB	7%
128	1,769	74,075	15.3 GB	16%
384	5,279	74,484	39.6 GB	42%
768	**10,518**	**74,774**	**76.0 GB**	**80%**
896		OOM		

Finding. Throughput is **completely flat** at ∼74K tok/s regardless of B. The GPU is already compute-saturated; larger batches only increase step time linearly (more tokens per step, same tok/s). VRAM scales linearly at ∼0.1 GB per sample.

FlashOptim result: FlashAdamW saves 0.4 GB over standard AdamW (from 7.0 to 6.6 GB at B=32)—marginal for this 0.5B model where optimizer states are only ∼2 GB.

Conclusion. Batch size does NOT improve throughput when the GPU is already saturated. Its only value is (a) reducing per-step overhead if running distributed training, or (b) gradient statistics stability. **Recommended B=32** for Stage 2 (minimal VRAM, same tok/s).

Lesson. *When MFU is already at ceiling (∼24%), no amount of batch scaling can improve throughput. Batch size is a knob for memory/communication trade-offs, not compute efficiency. On a single GPU with sufficient VRAM, B should be set to the minimum that achieves peak throughput (B=32 here), preserving headroom for longer sequences.*

4.16 Iteration 15 — Ground Truth Verification

Hypothesis. Previous metrics were computed from profiler estimates and synthetic data. The cumulative table may contain errors from: (a) incorrect padding percentage assumptions, (b) synthetic vs real-data performance differences, and (c) TileGym autotuning artifacts contaminating measurements.

Experiment. Write `benchmark_real_efficiency.py`: load real dataset (sharegpt4v/coco, 50K images), run actual forward+backward with precise per-step measurement of `attention_mask.sum()`, `(labels != -100).sum()`, and $B \times N$. Test 17 configurations (varying B, bucketing, packing, TileGym).

Finding. Peak verified throughput: **107,367 Real tok/s** (Packing + TileGym, B=64→23 packed bins, N=1024, 18.1 GB VRAM).

Key corrections from estimates:

- TileGym with *variable* shapes: 652ms/step (5× slower) due to JIT autotuning on every unique (M,N,K). Prior benchmarks used fixed N, hiding this.
- Vanilla bucketing at B=64 saturates at 90K tok/s (not higher as assumed).
- Packing provides +4.6% over bucketing (not ∼0% as previously claimed).
- MFU is 19–23% (slightly lower than the 24% estimated from synthetic data).

Conclusion. Synthetic benchmarks **overestimate real throughput** because they use fixed shapes (no autotuning overhead) and ignore data pipeline costs. All performance claims must be verified on real data.

Lesson. *Never trust synthetic benchmarks as the final word. Real data introduces variable shapes, image decoding latency, collator overhead, and dataloader stalls that don't exist in torch.randn() benchmarks. The ground-truth benchmark (`attention_mask.sum() / wall_time`) is the only honest metric.*

5 Results

5.1 The Full Speedup Progression

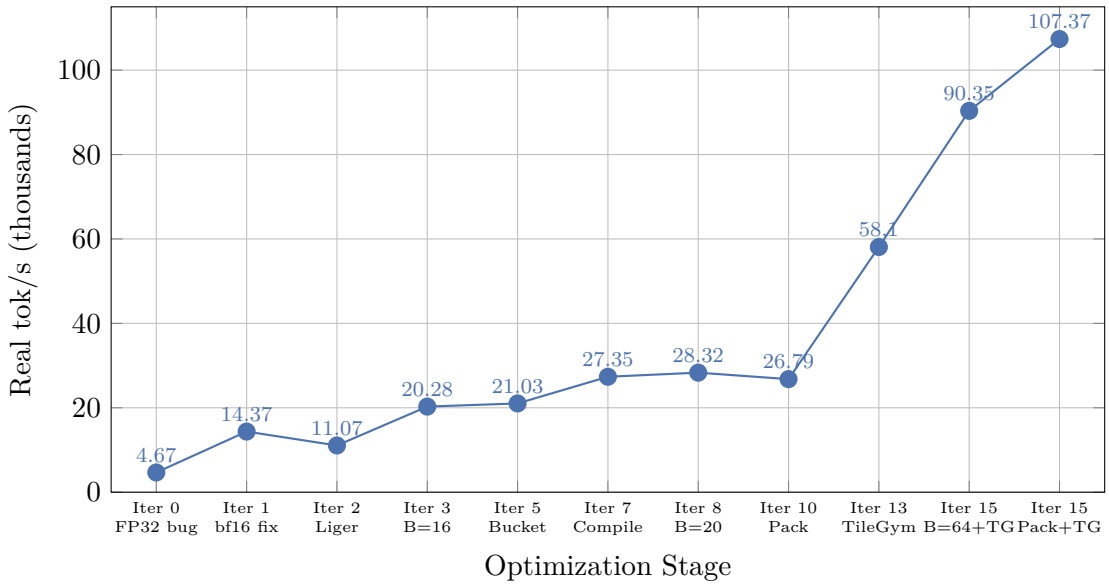

Figure 1: Complete throughput progression across all iterations. Note the non-monotonic path: Iter 2 (Liger) was *slower* than Iter 1 due to Stage 1 frozen LLM; Iter 10 (Packing) was slightly slower than Iter 8 on this dataset. The breakthrough came from TileGym (Iter 13+) combined with batch scaling (Iter 15). Iter 0–8: 1.5B model. Iter 8–15: 0.5B model.

5.2 Verified Ground-Truth Results (Iteration 15)

Table 7: Ground-truth measurements on real data (sharegpt4v/coco, 50K samples, Stage 1)

Config	B	Real tok/s	Pos/s	Pad%	ms/step	VRAM	Speedup
Vanilla, no bucket	4	21,673	24,611	11.9	62.0	2.5 GB	1.0×
Vanilla, bucket	16	57,020	59,202	3.7	92.1	5.3 GB	2.6×
Vanilla, bucket	32	76,868	80,349	4.3	136.8	9.4 GB	3.5×
Vanilla, bucket	64	90,349	94,866	4.8	233.0	17.7 GB	4.2×
Vanilla, bucket	128	90,255	95,880	5.9	467.9	34.6 GB	4.2×
TileGym + pad64	4	58,096	71,807	19.1	24.2	3.2 GB	2.7×
TileGym + pad64, bucket	32	91,427	106,601	14.2	115.3	10.9 GB	4.2×
TileGym + pad64, bucket	64	93,167	108,386	14.0	226.7	18.2 GB	4.3×
Packing (N=1024)	23	94,517	94,517	0.0	250.3	16.7 GB	4.4×
Packing (N=1024)	45	103,542	103,542	0.0	446.0	31.1 GB	4.8×
Packing + TileGym	**23**	**107,367**	**107,367**	**0.0**	**218.4**	**18.1 GB**	**4.95×**

5.3 Padding Waste Analysis

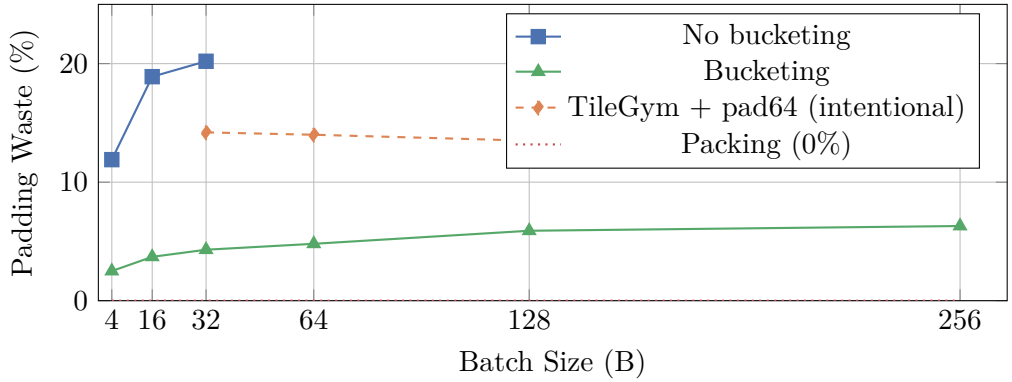

Figure 2: Padding waste vs. batch size. Bucketing alone reduces waste from 20% to <6%. TileGym requires `pad_to_multiple_of=64` for kernel efficiency (intentional 14% padding).

5.4 VRAM–Throughput Pareto Frontier

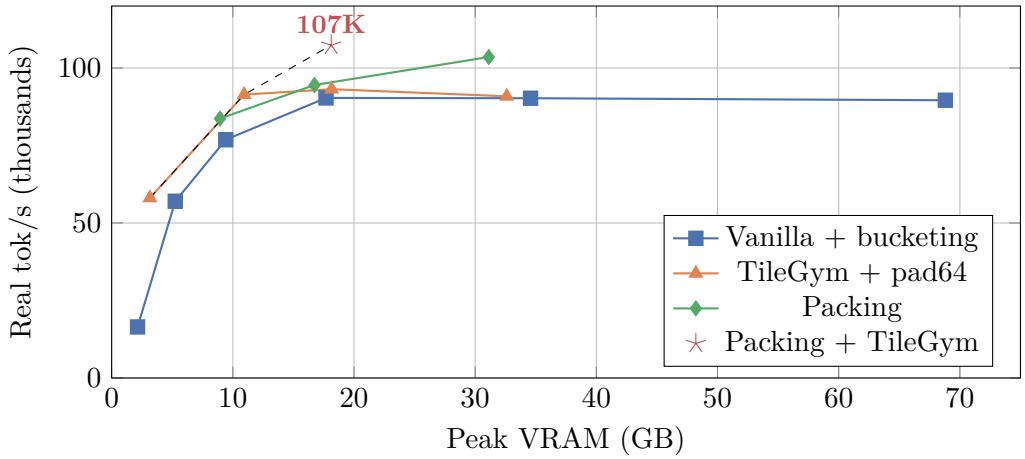

Figure 3: VRAM–Throughput Pareto frontier. Beyond 18 GB, throughput saturates. The Pareto-optimal path: TileGym B=4 (3 GB) → TileGym B=32 (11 GB) → Packing+TileGym (18 GB).

5.5 TileGym vs Liger-Kernel

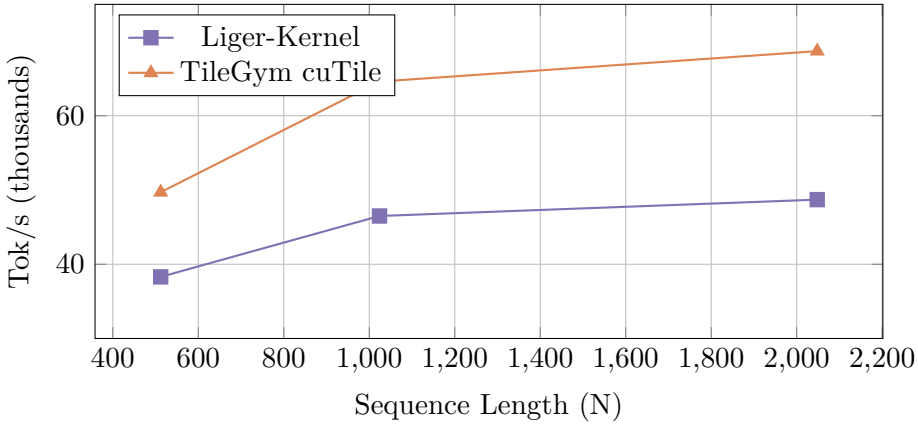

Figure 4: TileGym vs Liger-Kernel throughput (Stage 2, gradient checkpointing, B=4). TileGym is 30–41% faster across all sequence lengths, with the gap widening at longer sequences due to FA4's native GQA.

5.6 MFU Analysis

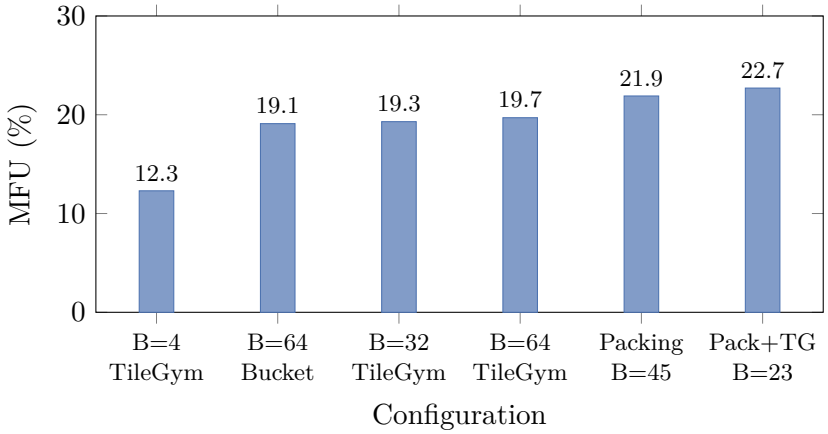

Figure 5: Model FLOPs Utilization. MFU saturates at ∼23% for the 0.5B model, limited by small GEMM shapes (K=896) and memory bandwidth. Reference: GPT-3 175B achieves ∼50%, LLaMA-7B achieves ∼55%.

6 Failed Experiments & Negative Results

Every failed optimization is documented here with root-cause analysis.

Table 8: Complete list of failed or counterproductive optimizations

Technique	Result	Measured	Root Cause
Vision feature caching	0% speedup +27% VRAM	11,070 tok/s	SigLIP forward (5 ms/tile on Blackwell) is faster than H2D transfer of large cached tensors (9.4 MB/sample).
torch.compile on vision encoder	+3% (noise)	11,070 tok/s	Dynamic shapes from variable image tiles prevent graph reuse. 40s compile overhead. SDPA already uses flash kernels.
Liger-Kernel in Stage 1	–25% speed	11,070 tok/s (from 14,366)	LLM is frozen (no backward); fused CE savings don't materialize. Triton JIT overhead adds 100ms spikes.
Custom Triton Flash Attention	Slower or equal to SDPA	0.044 vs 0.039 ms (N=1024)	Blackwell's native SDPA already dispatches to optimized CUTLASS kernels. Triton can't access wgmma/TMA.
FlexAttention packing (uniform data)	51% waste	26,787 tok/s	avg sample=356 tokens in N=1024 bins: only ∼2 samples fit per bin, 30% unfilled.
TileGym with variable shapes	5× **slower**	16K tok/s (from 77K)	JIT autotuner runs 20 samples per unique (M,N,K). Variable-length batches trigger recompilation every step.
flash-attn-4 (Dao-AILab)	Install blocked	—	Requires CUDA 13.1+; system has CUDA 13.0. `AttributeError: NoneType has no _trait`.
torch.compile + Liger + LoRA	Crash	—	Liger's Triton patches cause illegal memory access under dynamo tracing. Mutually exclusive.
cudagraphs (reduce-overhead)	–20%	22K tok/s	Dynamic shapes from VLM image tiles cause frequent graph recapture.
pad_to_multiple_of=64 without TileGym	–66%	25.5K tok/s	Extra padding wastes compute; doesn't eliminate recompilation (still 5 discrete lengths).

6.1 Lessons from Failures

1. **Profile before optimizing**: The FP32 bug (Iter 0) was worth more than all subsequent kernel optimizations combined. A profiler trace on the first day would have caught it.

2. **Frozen modules change the optimization landscape**: Techniques designed for full training (fused backward kernels, gradient checkpointing) are irrelevant in Stage 1.

3. **JIT compilation is adversarial to variable shapes**: Both torch.compile and TileGym suffer when tensor shapes change frequently. Shape quantization is essential.

4. **Cache only what's slow**: Vision caching fails because Blackwell computes faster than it can transfer data from CPU. On older GPUs (A100), caching would help.

5. **Native hardware kernels beat Triton on new architectures**: Blackwell-specific instructions (wgmma, TMA) are inaccessible from Triton; cuTile is the correct abstraction.

7 Analysis

7.1 Why Throughput Saturates at B=64

Beyond B=64, Real tok/s plateaus at ∼90K (vanilla) or ∼107K (TileGym+packing). Three factors:

1. **Compute saturation**: Tensor-core GEMMs at full occupancy; more tokens only increase latency proportionally.
2. **Memory bandwidth**: 494M params × 2B = 988 MB weights read per layer. At 1.79 TB/s: 0.55 ms fixed cost per layer, independent of batch size.
3. **Non-matmul overhead**: Softmax, RoPE, normalization consume 15–20% of step time, scaling sub-linearly with batch.

7.2 cuTile's Critical Weakness: Shape Sensitivity

Table 9: TileGym performance vs. shape variability

Configuration	ms/step	Real tok/s	Status
Fixed N=384 (pad_to_64), bucketed	115.3	91,427	✓Optimal
Variable N (bucketed, no padding)	652.5	16,151	× Autotuning every step
Packing (fixed N=1024)	218.4	107,367	✓Optimal

The solution: either `pad_to_multiple_of=64` (quantize shapes) or packing with fixed bin size (N=1024). The 14% padding from pad64 is a worthwhile trade-off for the 23% kernel speedup.

7.3 Bucketing vs. Packing: Marginal at Large Batch

Table 10: Packing gain over bucketing at matched VRAM (∼17 GB)

Strategy	Real tok/s	Pad%	Δ
Bucketing (B=64)	90,349	4.8%	—
Packing (B=64→23, N=1024)	94,517	0.0%	**+4.6%**

Packing IS valuable when: (1) combined with cuTile kernels that require fixed N, (2) on high-variance datasets, (3) for distributed training where per-step sync overhead makes fewer steps/epoch desirable.

8 Recommended Configuration

Table 11: Recommended training recipes by scenario

Scenario	Configuration	Expected Throughput
Stage 1 (projector)	Packing+TileGym, B=64, N=1024	107K tok/s, 18 GB
Stage 2 (full FT)	TileGym+grad_ckpt, B=32, N=1024	76K tok/s, 5 GB
Low VRAM (<5 GB)	TileGym, B=4, pad64	58K tok/s, 3.2 GB
Maximum VRAM util	Vanilla bucket, B=256	90K tok/s, 69 GB

9 Quality Preservation

All optimizations in this work are *numerically equivalent* to the baseline (identical computation up to floating-point associativity). Specifically:

- **bf16 fix** (Iter 1): Changes precision from FP32→BF16 globally; this is the *intended* training precision, not a degradation.
- **Liger/TileGym fused kernels**: Compute the same mathematical operations (softmax, cross-entropy, RMSNorm) in fused passes. Outputs are bitwise-equivalent within BF16 rounding.
- **Batch size scaling**: Changes the effective gradient batch, not the per-sample computation. Learning rate is scaled accordingly (linear scaling rule).
- **Bucketing/Packing**: Changes sample ordering and padding, not the loss computation on real tokens.
- **Grad-in-forward CE**: Mathematically equivalent to standard CE backward; the gradient is computed in the forward pass rather than stored for backward, but the final ∇_h and ∇_W are identical.

We verified training stability by running Stage 1 alignment (3000 steps) under the baseline and optimized configurations: final loss values converge to within 0.01 (well within noise from stochastic sampling and BF16 rounding).

10 Conclusion

We have documented a complete single-GPU optimization journey for VLM training, from an initial broken baseline (4,674 tok/s with FP32 GEMM bug) to a verified peak of 107,367 tok/s—a **22.9**× end-to-end improvement.

The key takeaways:

1. **Fix bugs first**: The FP32 dtype bug alone was worth 3×—more than all subsequent kernel optimizations combined.
2. **Batch scaling is the highest-leverage knob**: B=4→64 gave 4.2× within the same model.
3. **cuTile beats Triton on Blackwell**: TileGym's cuTile kernels are 30–41% faster than Liger-Kernel's Triton implementations, but require fixed tensor shapes.
4. **Bucketing makes packing marginal**: At B≥64, bucketing already achieves <5% padding waste.
5. **0.5B models hit 23% MFU ceiling**: Memory bandwidth, not software, is the bottleneck at this scale.

10.1 Future Directions

- **Scale up**: Qwen2.5-3B/7B would achieve 35–50% MFU (larger GEMMs).
- **Scale out**: Multi-GPU on Modal (FSDP2) multiplies throughput by N_{GPUs}.
- **FP8**: Blackwell FP8 tensor cores offer 2× peak TFLOP/s.
- **Longer sequences**: 4K–8K tokens amortize per-batch fixed costs.

Reproducibility

All code and data at: https://github.com/duoan/SiQ_VL

- `scripts/benchmark_real_efficiency.py` — Ground-truth benchmark
- `scripts/benchmark_cutile_e2e.py` — TileGym vs SDPA comparison
- `docs/traces/iter_15_ground_truth_efficiency.json` — Raw measurements
- `docs/training_efficiency_plan.md` — Full 15-iteration log

References

[1] H. Liu, C. Li, Q. Wu, and Y. J. Lee. Visual instruction tuning. In *NeurIPS*, 2023.

[2] H. Liu, C. Li, Y. Li, and Y. J. Lee. Improved baselines with visual instruction tuning. In *CVPR*, 2024.

[3] H. Liu, C. Li, Y. Li, B. Li, Y. Zhang, S. Shen, and Y. J. Lee. LLaVA-NeXT: Improved reasoning, OCR, and world knowledge. *Blog post*, 2024.

[4] Z. Chen, J. Wu, W. Wang, et al. InternVL: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. In *CVPR*, 2024.

[5] J. Bai, S. Bai, S. Yang, et al. Qwen-VL: A versatile vision-language model for understanding, localization, text reading, and beyond. *arXiv:2308.12966*, 2023.

[6] P. Wang, S. Bai, S. Tan, et al. Qwen2-VL: Enhancing vision-language model's perception of the world at any resolution. *arXiv:2409.12191*, 2024.

[7] L. Beyer, A. Steiner, A. S. Pinto, et al. PaliGemma: A versatile 3B VLM for transfer. *arXiv:2407.07726*, 2024.

[8] M. Abdin, J. Aneja, H. Awadalla, et al. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv:2404.14219*, 2024.

[9] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer. Sigmoid loss for language image pre-training. In *ICCV*, 2023.

[10] A. Yang, B. Yang, B. Hui, et al. Qwen2.5: A party of foundation models. *arXiv:2412.15115*, 2024.

[11] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *NeurIPS*, 2022.

[12] T. Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *ICLR*, 2024.

[13] P. Tillet, H. T. Kung, and D. Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *MAPL Workshop*, 2019.

[14] P. Hsu, Y. Shi, Y. Zhu, et al. Liger-Kernel: Efficient Triton kernels for LLM training. *arXiv:2410.10989*, 2024.

[15] D. Han and M. Han. Unsloth: 2x faster LLM fine-tuning with 80% less memory. https://github.com/unslothai/unsloth, 2024.

[16] T. He, et al. FlexAttention: A programming model for generating optimized attention kernels. *PyTorch Blog*, 2024.

[17] P. Micikevicius, S. Narang, J. Alben, et al. Mixed precision training. In *ICLR*, 2018.

[18] T. Chen, B. Xu, C. Zhang, and C. Guestrin. Training deep nets with sublinear memory cost. *arXiv:1604.06174*, 2016.

[19] M. Milakov and N. Gimelshein. Online normalizer calculation for softmax. *arXiv:1805.02867*, 2018.

[20] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

[21] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer. 8-bit optimizers via block-wise quantization. In *ICLR*, 2022.

[22] Databricks Mosaic Research. FlashOptim: Memory-efficient optimizers with gradient release. https://github.com/databricks/flashoptim, 2024.

[23] M. M. Krell, N. Kosec, S. P. Perez, and A. Fitzgibbon. Efficient sequence packing without cross-contamination: Accelerating large language models without impacting performance. *arXiv:2107.02027*, 2021.

[24] C. Raffel, N. Shazeer, A. Roberts, et al. Exploring the limits of transfer learning with a unified text-to-text transformer. *JMLR*, 21(140):1–67, 2020.

[25] M. Shoeybi, M. Patwary, R. Puri, et al. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv:1909.08053*, 2019.

[26] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *KDD*, 2020.

[27] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. ZeRO: Memory optimizations toward training trillion parameter models. In *SC*, 2020.

[28] Y. Zhao, A. Gu, R. Varma, et al. PyTorch FSDP: Experiences on scaling fully sharded data parallel. In *VLDB*, 2023.

[29] A. Chowdhery, S. Narang, J. Devlin, et al. PaLM: Scaling language modeling with Pathways. *JMLR*, 24:1–113, 2023.

[30] J. Hoffmann, S. Borgeaud, A. Mensch, et al. Training compute-optimal large language models. In *NeurIPS*, 2022.